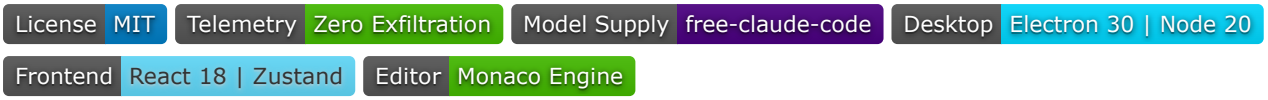
Ty-Code ⚡

> The Autonomous, Local-First AI Coding Desktop IDE.
> Built for senior software engineers, architects, and security-conscious teams who demand Cursor-level agentic velocity without cloud surveillance, vendor lock-in, or runaway token invoices.



Overview

is an open-source, air-gap-capable desktop AI integrated development environment built on the (the core engine behind VS Code), , , and .

Traditional cloud-centric AI coding tools (e.g., GitHub Copilot, Cursor, Windsurf) force engineering organizations into an untenable compromise: streaming proprietary codebases, internal API endpoints, and sensitive credentials over public networks to third-party multitenant cloud servers.

Ty-Code fundamentally eliminates this compromise. By orchestrating a , an intelligent , and operating-system-level hardware credential encryption, Ty-Code runs entirely under your control. It ships with a built-in zero-configuration free tier, connects directly to over 100+ commercial frontier models via Bring-Your-Own-Key (BYOK), and integrates with local open-weights engines (Ollama, vLLM) with mathematical workspace containment.

Core Philosophy

1. 🛡️ True Local-First Sovereignty (Zero-Telemetry)

Your code belongs to you. Ty-Code performs repository scanning, abstract syntax tree (AST) indexing, semantic drift detection, workspace diffing, and correction rule learning .

- No background analytics or event logging.
- No remote code snippet indexing.

- Direct outbound egress connects exclusively to your explicitly configured model endpoints.

2. 🔑 Bring-Your-Own-Key (BYOK) Freedom

Never pay a $20/month subscription markup for rate-limited API access. Provide your own direct API keys for OpenRouter, Anthropic, OpenAI, Groq, Cerebras, or Google AI Studio. Pay raw wholesale token rates directly to the model providers, or point Ty-Code at your on-premises vLLM / Ollama cluster for 100% air-gapped, zero-cost execution.

3. 🔒 Hardware-Backed OS Credential Encryption (`safeStorage`)

Plaintext `.env` secret storage is an enterprise vulnerability. Ty-Code seals all API tokens, session cookies, and OAuth refresh tokens using operating system cryptographic primitives:

- Hardware-sealed via Windows Data Protection API (DPAPI).
- Stored within the Apple Keychain Services architecture via 256-bit AES-GCM managed by the Secure Enclave.
- Mediated via `libsecret` binding to the system Secret Service (GNOME Keyring / KWallet).
- Credentials decrypt once per app boot into volatile memory, serving model dispatch requests with 0ms disk overhead.

Key Features Grid

```
+-------------------------------------------------------------------------------------------
|                                  TY-CODE CORE SYSTEM CAPABILITIES
+-----------------------------------+-----------------------------------+-------------------
| 🌐 Omni-Orchestrator Routing      | 🧠 Ty-Code Context Engine         | 🔍 Omni-Debugger
| • Auto-failover across providers  | • Universal Correction Memory     | • AST architectural drif
| • Rate-limit predictive backoff   | • Dynamic Auto-MCP tool routing   | • Target file style chec
| • Connection pool (keepAlive)     | • Progressive 10k file scanner    | • SQLite / DB profiling
+-----------------------------------+-----------------------------------+-------------------
| 🎨 M3E Visual UI Canvas           | 🤖 Enterprise Agentic Harness     | 🚀 Matrix Decode Engine
| • Project-native design extractor | • ReAct loop with Task Scoping    | • Decode-Frontier engine
| • Bidirectional Monaco sync       | • Auto-Compaction at 95% context  | • 140 char/sec stream
| • Component isolation sandbox     | • 3-Tier Terminal Sandbox Gate    | • Gated Markdown swap
+-----------------------------------+-----------------------------------+-------------------
```

Feature Deep-Dive

Subsystem	Architecture & Enterprise Impact
	Dynamic Directed Acyclic Graph (DAG) routing engine. Automatically detects 429 rate limits and 5xx upstream outages, seamlessly pivoting mid-turn across OpenRouter, Groq, Anthropic, and local Ollama without dropping context. Employs TCP/TLS keepAlive connection pooling to shave 180ms off first-byte latency.
Ty-Code Context Engine	Features Universal Correction Memory (The Learning Loop): when you correct the AI in Chat, the Omni-Debugger, or the M3E Canvas, the rule is saved to a local Write-Ahead-Logged SQLite database and dispatched live across all three surfaces. Features Auto-MCP: dynamically maps tool schemas on-demand without context window exhaustion.
	Specialized multi-threaded static analysis worker pool (`arch-worker.ts`). Continuously checks code mutations for architectural drifts, forbidden patterns (e.g., legacy `var` usage, unescaped raw SQL, missing repository layers), and runtime exceptions.
M3E UI Canvas	An interactive design-to-code canvas built right into your IDE. Automatically extracts your project's active design system (Tailwind tokens, CSS variables, typography), letting you visually manipulate UI layouts with instant, bidirectional Monaco editor code synchronization.
	Autonomous ReAct execution engine with structured Task Scoping, Todo checklist tracking, and autonomous compaction when context capacity hits 95%. Safeguarded by a (`Restricted` , `Always Ask` , `Always Proceed`) with rigorous PTY environment scrubbing.

The Zero-Friction Free Tier & One-Key Premium

Ty-Code provides the smoothest onboarding experience in the developer tools industry:

1. Zero-Friction Community Free Tier (Out of the Box)

Launch Ty-Code and start coding immediately. Out of the box, Ty-Code connects to our distributed Cloudflare Worker edge gateway (`api.ty-code.com`), providing instant access to fast, capable open-weights models (such as Llama 3.3 70B and Qwen 2.5 Coder):

- ❌ No account registration required.

- ❌ No credit card required.
- ❌ Zero environment configuration.
- ✅ Instant code generation, debugging, and chat.

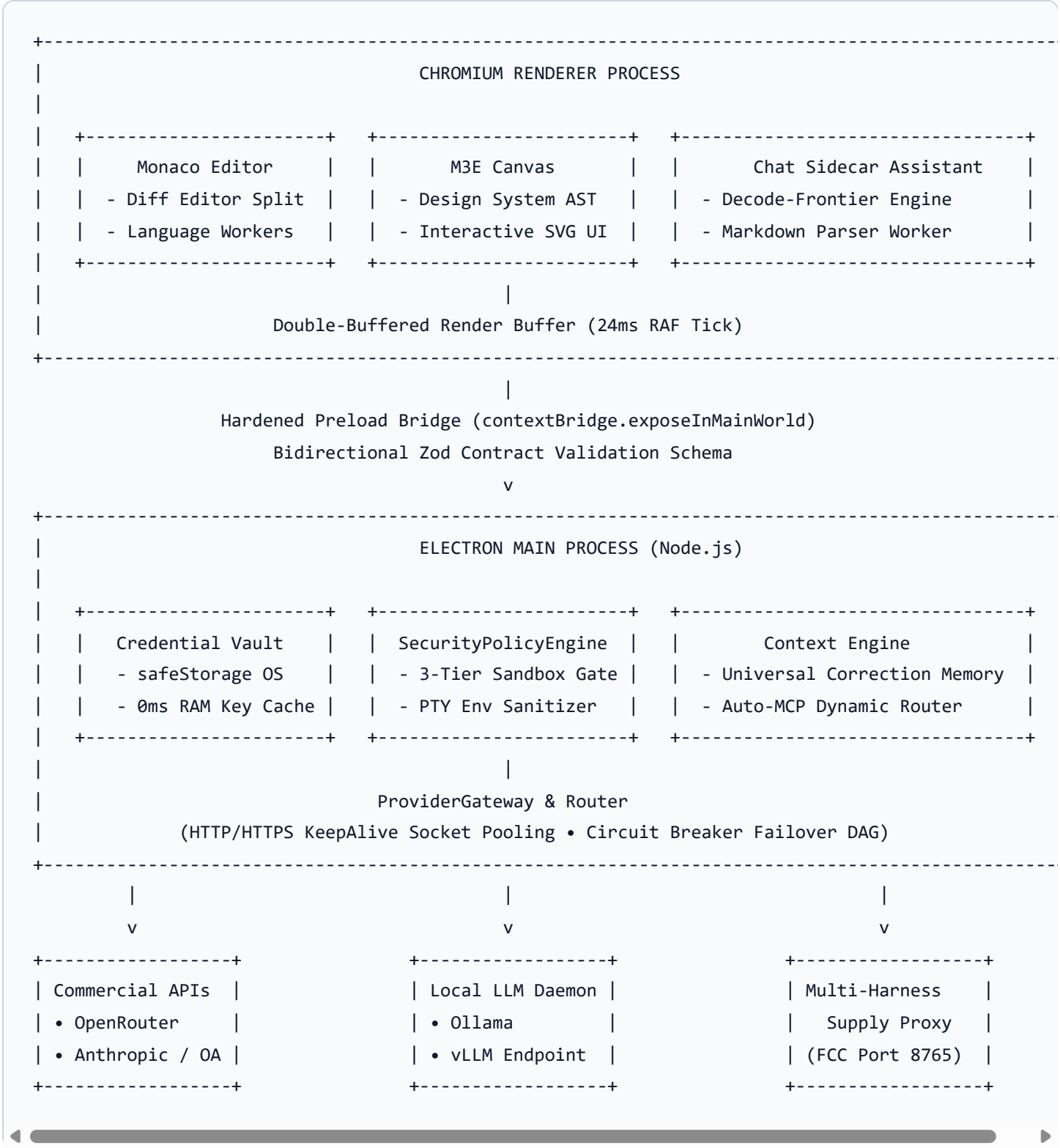
2. "One-Key Premium" Upgrade

When your engineering tasks require frontier commercial intelligence (Claude 3.5 Sonnet, GPT-4o, Gemini 1.5 Pro, DeepSeek V3):

1. Open Settings (`Ctrl+,` or `Cmd+,`) $\rightarrow$.
2. Paste a single OpenRouter API key.
3. Instantly unlock 100+ state-of-the-art models with native budget caps, token telemetry, and automatic model rotation.

Architecture Overview

Ty-Code maintains strict architectural isolation between presentation, native operating system resources, and external model dispatch:



The 24ms Double-Buffered Token Stream & Decode-Frontier Engine

High-velocity models (like Groq or Cerebras) stream tokens at 300–1,500 tokens/sec. Naive React rendering locks the main thread and causes severe layout thrashing. Ty-Code solves this with a :

- Raw SSE chunks collect in an off-screen string accumulator.
- The render buffer flushes to React state exactly once every 24ms.
- Renders an animated glyph-scramble band that sweeps forward at 140 chars/sec during streaming and 600 chars/sec on completion. Markdown AST parsing and code syntax

highlighting are offloaded to dedicated Web Workers in the background, cleanly swapping in once the decode sweep settles with .

Directory Layout

```
ty-code/
├── docs/                          # Technical architecture, security models & whitepapers
│   ├── ENTERPRISE-WHITEPAPER.md    # 10-page CISO/CTO Enterprise Architecture whitepaper
│   ├── EXTENSION-SECURITY-MODEL.md # VS Code extension capabilities & containment model
│   ├── FCC-INTEGRATION-ARCHITECTURE.md # Multi-harness supply architecture specifications
│   └── traces/                    # Continuous automated compliance & security traces
├── src/
│   ├── main/                      # Electron Main Process (Node.js LTS)
│   │   ├── agent/                 # ReAct agent loop, task scoping & tool executors
│   │   ├── providers/             # Multi-protocol ProviderGateway (SSE, NDJSON, KeepAlive)
│   │   ├── services/              # Credential vault, context engine, debugger orchestrator
│   │   └── ipc/                   # Zod-validated IPC channel registry
│   ├── preload/                   # Secure Preload Execution Context
│   │   ├── index.ts               # contextBridge method exposure
│   │   └── types.ts               # Shared IPC TypeScript contracts
│   └── renderer/                  # Chromium UI Layer (React 18 + Zustand + Monaco)
│       └── src/
│           ├── components/        # Monaco editor, M3E Canvas, Chat Sidecar, Terminal
│           ├── state/             # Zustand state stores (workspace, chat, audit)
│           └── workers/           # Web Workers for background Markdown & Syntax Highlighting
├── website/                       # Cloudflare Pages release portal & static docs
├── package.json
└── tsconfig.json
```

📥 Downloads & Binary Releases

Download verified, pre-built production binaries for your operating system:

Platform	Architecture	Package Format	Direct Download
	x64	NSIS Installer	`Ty-Code-Setup-1.0.0.exe`
Windows	x64	Portable Archive	`Ty-Code-1.0.0-win.zip`
	Universal	Disk Image (.dmg)	`Ty-Code-1.0.0.dmg`
Linux	x64	Universal AppImage	`Ty-Code-1.0.0.AppImage`
	x64	Debian / Ubuntu (.deb)	`ty-code_1.0.0_amd64.deb`
Linux	x64	RedHat / Fedora (.rpm)	`ty-code-1.0.0.x86_64.rpm`
	All	SHA-256 Signatures	`SHA256SUMS.txt`

Verify release binary integrity:

```
sha256sum -c SHA256SUMS.txt --ignore-missing
```

Getting Started (Local Development)

Prerequisites

- `v20.0.0` or higher (tested on Node.js v24 LTS)
- `v10.0.0` or higher
-

1. Clone & Install Dependencies

```
git clone https://github.com/JTyrell/Ty-Code.git
cd Ty-Code
npm ci
```

2. Run in Development Mode

Starts the Vite renderer server and esbuild main/preload watch compiler concurrently:

```
npm run dev
```

3. Verify System Invariants & Anti-Phantom Guards

Ty-Code includes an automated suite of enterprise security guards that audit the codebase against mock elimination, secret privacy, and attribution:

```
npm run guards
```

4. Build Production Binaries

Compile the main bundle, preload script, and optimized production renderer:

```
npm run build
```

To package native desktop installers for your current operating system:

```
npm run package:win   # Windows NSIS Installer & Portable ZIP
npm run package:mac   # macOS Universal DMG
npm run package:linux # Linux AppImage, DEB, and RPM
```

Acknowledgments & Open Source Credits

Ty-Code stands on the shoulders of the global open-source community. We express our profound gratitude to the maintainers and contributors of Monaco Editor, Electron, React, Zustand, Vite, and esbuild.

🌟 Special Attribution: free-claude-code (FCC)

We formally acknowledge and credit the groundbreaking engineering established by the project:

> Repository: https://github.com/Alishahryar1/free-claude-code
> Upstream Attribution String: *Free-model supply layer powered by free-claude-code (FCC) — AGPL-3.0 / MIT — https://github.com/Alishahryar1/free-claude-code*

Ty-Code's —which unlocks access to over across 11 distinct development environments (including Claude Code, Codex, VS Code Extension APIs, Pi, OpenCode, Cline, Aider, Hermes, DeepSeek, Grok-Build, and Mistral)—was deeply inspired by and architected upon the methodologies pioneered by the FCC repository.

By adapting FCC's mock-free binary loopback patterns, ToS safety compliance categorizations, and subscription OAuth device flows into Ty-Code's hardened desktop sandbox, we have bridged community-driven open-source innovation with enterprise-grade security. This integration stands as a shining testament to how collaborative open-source engineering drives modern developer tools forward.

Enterprise Whitepaper & Formal Specifications

For enterprise procurement committees, CISOs, and enterprise architects evaluating Ty-Code for corporate-wide deployment:

- 📄 docs/ENTERPRISE-WHITEPAPER.md
- 🔒 docs/EXTENSION-SECURITY-MODEL.md
- 🌐 docs/FCC-INTEGRATION-ARCHITECTURE.md
- 📊 docs/traces/open-perf-trace.json

License

© 2026 Ty-Code Project • Version 1.0.0 Stable Release.
Licensed under the MIT License.
The integrated multi-harness free supply adapter layer interfaces with upstream tooling under AGPL-3.0 / MIT.